

```
// OSSP Skill 12 - Pipes and Multiple Pipelines
// Demonstrates pipe(), fork(), dup2(), descriptor closing,
// synchronization, and a pipeline with multiple processes.

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <errno.h>

static void die(const char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

int main(void) {
    int p1[2], p2[2];
    pid_t a, b, c;

    printf("=== Skill 12: Multiple Pipes ===\n");
    printf("Running: printf 'apple\\nbanana\\napple\\n' | grep apple | wc -l\n");

    if (pipe(p1) == -1 || pipe(p2) == -1)
        die("pipe");

    a = fork();
    if (a == -1) die("fork");
```

```
if (a == 0) {
    dup2(p1[1], STDOUT_FILENO);
    close(p1[0]); close(p1[1]);
    close(p2[0]); close(p2[1]);

    execlp("printf", "printf", "apple\nbanana\napple\n", NULL);
    perror("printf");
    _exit(1);
}

b = fork();
if (b == -1) die("fork");

if (b == 0) {
    dup2(p1[0], STDIN_FILENO);
    dup2(p2[1], STDOUT_FILENO);

    close(p1[0]); close(p1[1]);
    close(p2[0]); close(p2[1]);

    execlp("grep", "grep", "apple", NULL);
    perror("grep");
    _exit(1);
}

c = fork();
if (c == -1) die("fork");
```

```

    if (c == 0) {
        dup2(p2[0], STDIN_FILENO);

        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);

        execlp("wc", "wc", "-l", NULL);
        perror("wc");
        _exit(1);
    }

    close(p1[0]); close(p1[1]);
    close(p2[0]); close(p2[1]);

    waitpid(a, NULL, 0);
    waitpid(b, NULL, 0);
    waitpid(c, NULL, 0);

    printf("All pipeline processes completed.\n");
    return 0;
}
|

```

```

sai_pavani@Sai-Pavani:~/OS_SKILL$ nano skill_12.c.c
sai_pavani@Sai-Pavani:~/OS_SKILL$ gcc skill_12.c.c -o skill_12
sai_pavani@Sai-Pavani:~/OS_SKILL$ ./skill_12
=== Skill 12: Multiple Pipes ===
Running: printf 'apple\nbanana\napple\n' | grep apple | wc -l
2
All pipeline processes completed.
sai_pavani@Sai-Pavani:~/OS_SKILL$ |

```